OWASP A01:2021 – Broken Access Control

Vertical Privilege Escalation Master Cheatsheet

A comprehensive engineering guide for discovering, analyzing, and mitigating vertical privilege escalation and Role-Based Access Control (RBAC) bypass vulnerabilities.

1. Core Concept & Root Cause

Vertical Escalation: Occurs when a user with low-privilege rights (Standard User / Guest) successfully accesses functionalities, sensitive endpoints, or administrative actions restricted strictly to higher-privilege roles (Administrators).

Root Cause: Relying on client-side concealment (hiding administrative links or UI elements) without enforcing strict server-side authorization checks for every single request.

2. Methodology & Attack Steps

- **Step 1 (Authentication):** Log in with a standard low-privilege account (`Standard User`). *(Note: In some misconfigurations, administrative endpoints are publicly accessible without any authentication).*
- **Step 2 (Disclosed Metadata):** Inspect `/robots.txt`, sitemaps, and technical documentation for administrative paths hidden from search engines.
- **Step 3 (Source Code Analysis):** Inspect the source code (`Ctrl + U`) to find developer comments, inline scripts, or embedded JavaScript (`.js`) files revealing hidden administrative API endpoints.
- **Step 4 (Fuzzing & Path Discovery):** Execute automated path fuzzing targeting common administrative endpoints if no direct paths are exposed in the UI.

3. Reconnaissance & Discovery Tools

LinkFinder:
Automatically parses JavaScript files to discover hidden API endpoints, routes, and forgotten developer endpoints.

JS-PathHunter:
Extracts inline comments, legacy routes, and internal endpoints using fast regex pattern matching.

Gobuster:
High-speed directory & file brute-forcer using wordlists to identify hidden endpoints yielding status codes like `200 OK` or `403 Forbidden`.

ffuf (Fast Fuzz):
Ultra-fast fuzzing tool to replace the FUZZ keyword across URLs, HTTP headers, or request payloads.

4. Role Mapping Matrix

Role Level	Intended Privilege	Target System Behavior	Testing & Verification Vector
Unauthenticated	Public Access	Access restricted to public landing pages & assets.	Direct URL request to admin endpoints without session cookies.
Standard User	Low Privilege	Manage personal account details and standard features.	Submit admin-level API requests using a valid low-privilege session.
Administrator	Full Control	Full user management, site configuration, & data deletion.	Compare endpoint response behavior between Admin and Standard User sessions.

5. Advanced Bypass Vectors (API, Headers & JWT)

A. HTTP Header Manipulation:
Inject custom headers like `X-Original-URL: /admin`, `X-Rewrite-URL: /admin`, or spoof internal IPs using `X-Forwarded-For: 127.0.0.1` to bypass reverse proxy/WAF rules.

B. HTTP Method Switching:
If an admin endpoint blocks `POST /admin/user/delete`, attempt executing the request using `GET`, `PUT`, or `PATCH`.

C. JSON Response Manipulation & Mass Assignment:
Tamper client-side roles by altering login responses (e.g., `"isAdmin": true`) or injecting privileged parameters like `"role": "admin"` in profile update POST bodies.

D. JWT Misconfigurations:
Modify claims in the token payload from `"role": "user"` to `"role": "admin"`, test `"alg": "none"` signature bypasses, or crack weak secret keys.

6. Automated Access Control Testing (Burp Autorize)

Burp Suite Extension — Autorize: The industry-standard tool for authorization testing.
It operates by allowing you to browse the target web application using a high-privilege account (Admin) while automatically repeating all HTTP requests in the background using a low-privilege session cookie and an unauthenticated request. It compares response status codes and byte lengths to highlight authorization flaws instantly.

7. Secure Coding & Remediation Guidelines

Enforce strict server-side authorization checks for every single endpoint request (Server-Side Enforcement):

❌ Vulnerable Pattern (Missing Function-Level Access Control):

```
@PostMapping("/admin/delete-user")
public ResponseEntity
        deleteUser(@RequestParam String userId) { // Server executes deletion relying solely on URL path
matching! userService.deleteById(userId); return ResponseEntity.ok().build(); }
```

✅ Secure Pattern (Centralized Role-Based Access Control - RBAC):

```
@PostMapping("/admin/delete-user")
@PreAuthorize("hasRole('ROLE_ADMIN')") // Centralized security annotation check public ResponseEntity
        deleteUser(@RequestParam String userId, Authentication auth) { if (auth == null ||
!auth.getAuthorities().contains(new SimpleGrantedAuthority("ROLE_ADMIN"))) { throw new
AccessDeniedException("403 Forbidden: Insufficient Permissions"); } userService.deleteById(userId); return
ResponseEntity.ok().build(); }
```